Programming with Python

Lecture 4: Data Structures and Collections II

Instructor: Chia-Heng Yen

Course Roadmap

- Fundamentals & Environment
  - Introduction to Python and Development Environment Setup
  - Flow Control and Basic Syntax
  - Data Structures and Collections
- Advanced Programming
  - Modular and Functional Programming
  - File Handling and Exception Handling
  - Object-Oriented Programming
  - Algorithm Implementation in Python
- User Interface Development
  - GUI Programming with Tkinter and PyQt
- Data Processing & Analysis
  - Web Scraping (requests / BeautifulSoup)
  - Data Analysis Fundamentals
  - Data Visualization
- AI & ML Applications
  - Introduction to Machine Learning with Applications
  - AI-Related Library in Python Applications

Recap: Collections I

- Four collections: list, tuple, set, dict
  - pick one by order, mutability, duplicates, and lookup by key
- Variables are name tags on objects
  - mutable objects change in place; immutable ones are replaced
- Only hashable (immutable) values can be set items or dict keys
- Membership test: set / dict O(1) on average, list O(n)
- Strings are immutable: build them with join(), not += in a loop

Outline

- Dict in depth: the key-value workhorse
  - hash tables, views, merging, counting / grouping patterns
- Strings: immutable Unicode text
  - C char arrays vs Python str, encoding, searching, cleaning
- String formatting
  - f-strings and the format-spec mini-language

Dict: the Key-Value Workhorse

Dictionary Basics

- What are Dictionaries?
  - Dictionaries are Key-Value Mapping

- Key Characteristics
  - Ordered (有序): Maintains insertion order (Python 3.7+)
  - Mutable (可變): Can be modified after creation
  - Keys must be unique and hashable
  - Fast lookup (快速查找): O(1) average time complexity

Dictionary Creation

- Creation Methods

```python
# Method 1: Curly braces with key:value pairs
student = {"name": "Alice", "age": 20, "major": "CS"}
numbers = {1: "one", 2: "two", 3: "three"}
mixed = {"string_key": 42, 1: "number_key", (1,2): "tuple_key"}

# Method 2: dict() constructor
person = dict(name="Bob", age=25, city="Taipei")
from_list = dict([("a", 1), ("b", 2), ("c", 3)])
from_zip = dict(zip(["x", "y", "z"], [1, 2, 3]))

# Method 3: Dictionary comprehension
squares = {x: x**2 for x in range(5)}          # {0: 0, 1: 1, 2: 4, 3: 9, 4: 16}

# Method 4: fromkeys() class method
default_dict = dict.fromkeys(["a", "b", "c"], 0)      # {'a': 0, 'b': 0, 'c': 0}

# Empty dictionary
empty_dict = {}
empty_dict2 = dict()
```

```python
names = ["Alice", "Bob", "Charlie"]
ages = [25, 30, 35]
paired = zip(names, ages)
print(list(paired))
# [('Alice', 25), ('Bob', 30), ('Charlie', 35)]
```

Essential Dictionary Operations

- Accessing and Modifying Values

```
student = {"name": "Alice", "age": 20, "major": "CS"}

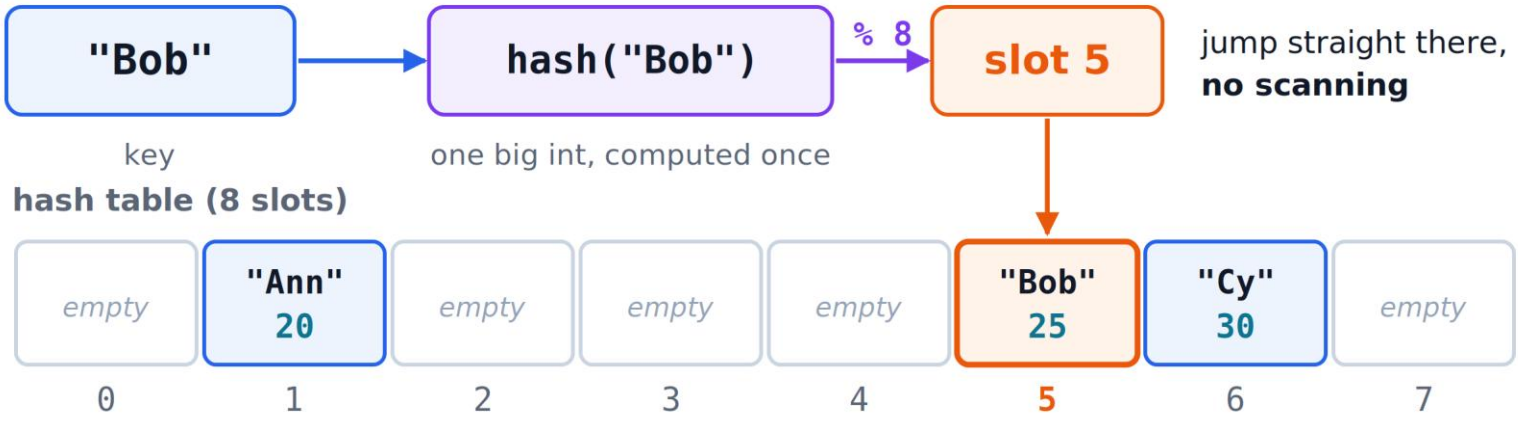
# Access values
name = student["name"]                  # "Alice" (raises KeyError if not found)
age = student.get("age")                # 20 (returns None if not found)
grade = student.get("grade", "A")       # "A" (default value if not found)

# Modify values
student["age"] = 21                     # Update existing key
student["grade"] = "A"                  # Add new key
student.update({"gpa": 3.8, "year": 2}) # Add multiple keys

# Remove values
del student["year"]                     #Remove key (raises KeyError if not found)
removed_grade = student.pop("grade")    # Remove and return value
last_item = student.popitem()           # Remove and return last (key, value) pair
```

How a Dict Works: the Hash Table

- hash(key) picks a slot, so lookup does not scan every item: O(1) average
- Equal keys must have equal hashes, so keys must never change
- That is why a list key raises TypeError



Guess the Output: Which Keys Survive?

- Three different-looking keys go in. What comes out?

```
d = {1: "a", 1.0: "b", True: "c"}
print(d)          # ?
print(len(d))     # ?
```

Answer **{1: 'c'}** **and** **1**

1 == 1.0 == True with equal hashes, so they are ONE key: the first key object is kept, the last value wins.

Essential Dictionary Operations

- Dictionary Methods

```
# Get all keys, values, and items
keys = student.keys()          # dict_keys(['name', 'age', 'major'])
values = student.values()      # dict_values(['Alice', 21, 'CS'])
items = student.items()        # dict_items([('name', 'Alice'), ('age', 21), ...])

# Convert to lists if needed
key_list = list(student.keys())
value_list = list(student.values())

# Check membership
has_name = "name" in student                # True (checks keys by default) O(?)
has_alice = "Alice" in student.values()     # True (check in values)  O(?)

# Clear all items
student.clear()                # {}
```

Looping Over a Dict (Without Breaking It)

- Loop over items() to get key and value together
- Adding or deleting keys while looping raises RuntimeError

```
scores = {"Ann": 92, "Bob": 55, "Cy": 78}
for name, s in scores.items():
    print(f"{name}: {s}")
# → Ann: 92
# → Bob: 55
# → Cy: 78
```

✗ Wrong

```
for name in scores:                # RuntimeError: dictionary
    if scores[name] < 60:          # changed size during iteration
        del scores[name]
```

✓ Right

```
for name in list(scores):          # loop over a copy of the keys
    if scores[name] < 60:
        del scores[name]
print(scores)                      # → {'Ann': 92, 'Cy': 78}
```

Dictionary Comprehensions and Merging

- Dictionary Comprehensions

```
# Basic syntax
# {key_expression: value_expression for item in iterable}

# Create squares dictionary
squares = {x: x**2 for x in range(6)} # {0: 0, 1: 1, 2: 4, 3: 9, 4: 16, 5: 25}
# With conditions
even_squares = {x: x**2 for x in range(10) if x % 2 == 0}
# Transform existing dictionary
prices = {"apple": 1.5, "banana": 0.8, "orange": 2.0}
expensive = {fruit: price for fruit, price in prices.items() if price > 1.0}
# String manipulation
text = "hello world"
char_count = {char: text.count(char) for char in set(text) if char != ' '}
# Swap keys and values
original = {"a": 1, "b": 2, "c": 3}
swapped = {value: key for key, value in original.items()} # {1: 'a', 2: 'b', 3: 'c'}
```

Dictionary Comprehensions and Merging

- Dictionary Merging

```
dict1 = {"a": 1, "b": 2}
dict2 = {"b": 3, "c": 4}
dict3 = {"d": 5, "e": 6}

# Method 1: update() method (modifies original)
result1 = dict1.copy()
result1.update(dict2)                                  # {"a": 1, "b": 3, "c": 4}
# Method 2: Dictionary unpacking (Python 3.5+)
merged = {**dict1, **dict2, **dict3}                   # {"a": 1, "b": 3, "c": 4, "d": 5, "e": 6}
# Method 3: Union operator (Python 3.9+)
merged_new = dict1 | dict2 | dict3                     # {"a": 1, "b": 3, "c": 4, "d": 5, "e": 6}
# Merge with custom logic
def merge_with_sum(d1, d2):
        result = d1.copy()
        for key, value in d2.items():
                result[key] = result.get(key, 0) + value
        return result
sum_merged = merge_with_sum({"a": 1, "b": 2}, {"b": 3, "c": 4})     # {"a": 1, "b": 5, "c": 4}
```

Dictionary Comprehensions and Merging

- Dictionary Merging

```
# Merge with list values
def merge_lists(d1, d2):
        result = {k: v.copy() for k, v in d1.items()}   # copy inner lists too
        for key, value in d2.items():
                if key in result:
                        result[key].extend(value)
                else:
                        result[key] = value.copy()
        return result

list_dict1 = {"fruits": ["apple", "banana"], "colors": ["red"]}
list_dict2 = {"fruits": ["orange"], "colors": ["blue", "green"], "numbers": [1, 2]}
merged_lists = merge_lists(list_dict1, list_dict2)
# {"fruits": ["apple", "banana", "orange"], "colors": ["red", "blue", "green"], "numbers": [1, 2]}
```

Pattern: Counting

- The hand-written get(k, 0) + 1 loop works
- collections.Counter is the idiom: a dict built for tallying

```
text = "hello world hello python"
word_count = {}
for word in text.split():
        word_count[word] = word_count.get(word, 0) + 1
# Result: {'hello': 2, 'world': 1, 'python': 1}

# The idiom: collections.Counter
from collections import Counter
wc = Counter(text.split())
print(wc == word_count)          # True
print(wc.most_common(1))         # [('hello', 2)]
print(wc["java"])                # 0 (missing key gives 0, no KeyError)
```

Pattern: Grouping

- setdefault(k, []) inserts [] if k is missing, then returns it

```
# Grouping data
students = [
        ("Alice", "CS"), ("Bob", "Math"), ("Charlie", "CS")
]
# Group by major
by_major = {}
for name, major in students:
        if major not in by_major:
                by_major[major] = []
        by_major[major].append(name)
# Using setdefault()
by_major2 = {}
for name, major in students:
        by_major2.setdefault(major, []).append(name)
print(by_major2)        # {'CS': ['Alice', 'Charlie'], 'Math': ['Bob']}
```

Pattern: Grouping

- defaultdict(list) creates the default on first access

```
students = [
        ("Alice", "CS"), ("Bob", "Math"), ("Charlie", "CS")
]
# defaultdict: a missing key gets a new [] automatically
from collections import defaultdict
by_major2 = defaultdict(list)
for name, major in students:
        by_major2[major].append(name)
print(dict(by_major2))                  # {'CS': ['Alice', 'Charlie'], 'Math': ['Bob']}

# defaultdict(int): a missing key starts at 0, a ready-made counter
counts = defaultdict(int)
for word in "hello world hello".split():
        counts[word] += 1
print(dict(counts))                     # {'hello': 2, 'world': 1}
```

Pattern: Grouping

- Invert a dict: group keys by their value

```
# Inverting dictionary
original = {"a": 1, "b": 2, "c": 1, "d": 3}
inverted = {}
for key, value in original.items():
        inverted.setdefault(value, []).append(key)
# Result: {1: ['a', 'c'], 2: ['b'], 3: ['d']}
```

Guess the Output: fromkeys with a List

- One append, two keys. What does d look like?

```
d = dict.fromkeys(["a", "b"], [])
d["a"].append(1)
print(d)        # ?
```

Answer `{'a': [1], 'b': [1]}`

Both keys tag the SAME list (name tags, Part I). Fix: d = {k: [] for k in keys}

Pattern: Dict as a Lookup Table

- Replace a long if / elif chain with a dict lookup
- get() with a default handles invalid input

```
# Dictionary as a switch statement
def process_grade(letter_grade):
        grade_map = {
                "A": (4.0, "Excellent"),
                "B": (3.0, "Good"),
                "C": (2.0, "Average"),
                "D": (1.0, "Below Average"),
                "F": (0.0, "Fail")
        }
        gpa, description = grade_map.get(letter_grade, (0.0, "Invalid Grade"))
        return {"gpa": gpa, "description": description}

result = process_grade("A") # {'gpa': 4.0, 'description': 'Excellent'}
```

Pattern: Dict as a Dispatch Table

- Map each case to a function instead of a long if / elif chain
- Like a C function-pointer table, but keyed by any hashable value

C

```c
// C: array of function pointers, indexed by an int
int (*ops[])(int, int) = {add, sub, mul};
```

Python

```python
import operator
ops = {"+": operator.add, "-": operator.sub,
       "*": operator.mul, "/": operator.truediv}
def calc(a, op, b):
    func = ops.get(op)
    if func is None:
        raise ValueError(f"unknown operator {op!r}")
    return func(a, b)
print(calc(6, "*", 7))    # → 42
print(calc(1, "/", 4))    # → 0.25
```

Nested Dicts: JSON-Shaped Data

- Chain [] for paths you know exist; chain get() when a level may be missing
- When the shape is fixed, a class is clearer (see the OOP lecture)

```
db = {
    "001": {"name": "Ann", "gpa": 3.8,
            "courses": ["Python", "Algorithms"]},
    "002": {"name": "Bob", "gpa": 3.6, "courses": []},
}
print(db["001"]["courses"][0])          # → Python
db["002"]["courses"].append("Math")     # change in place
print(db["002"]["courses"])             # → ['Math']
print(db.get("003", {}).get("name", "N/A"))    # → N/A
```

Copying Nested Dicts

- copy() is shallow: inner lists and dicts are still shared (Part I)
- copy.deepcopy() gives fully independent nested data

```
import copy
orig = {"Ann": {"courses": ["Python"]}}
shallow = orig.copy()
deep = copy.deepcopy(orig)
orig["Ann"]["courses"].append("C")
print(shallow["Ann"]["courses"])    # → ['Python', 'C']
print(deep["Ann"]["courses"])       # → ['Python']
```

List of Records vs Dict Keyed by ID

- Finding an id in a list is O(n) every time
- Build a dict index once, then each lookup is O(1)

```
students = [{"id": "001", "name": "Ann"},
            {"id": "002", "name": "Bob"}]
for s in students:                      # O(n) search
    if s["id"] == "002":
        print(s["name"])                 # → Bob
by_id = {s["id"]: s for s in students}   # build once
print(by_id["002"]["name"])              # → Bob   O(1)
print(by_id["001"] is students[0])       # → True   same object
```

Time Complexity Comparison

Operation	List 串列	Tuple 字元組	Set 集合	Dict 字典	Notes
Access	O(1)	O(1)	N/A	Avg. O(1) Worst O(n)	透過索引/鍵存取
Search	O(n)	O(n)	Avg. O(1) Worst O(n)	Avg. O(1) Worst O(n)	in運算子
Insert	O(1)~O(n)	N/A	Avg. O(1) Worst O(n)	Avg. O(1) Worst O(n)	位置相關
Delete	O(1)~O(n)	N/A	Avg. O(1) Worst O(n)	Avg. O(1) Worst O(n)	位置相關
Length	O(1)	O(1)	O(1)	O(1)	len()函數

Strings: Immutable Unicode Text

Strings: C vs Python

- C: a char array ending in '\0'; you manage length and memory
- Python: an immutable sequence of Unicode characters; len() is O(1)
- No separate char type: "a" is just a string of length 1

C

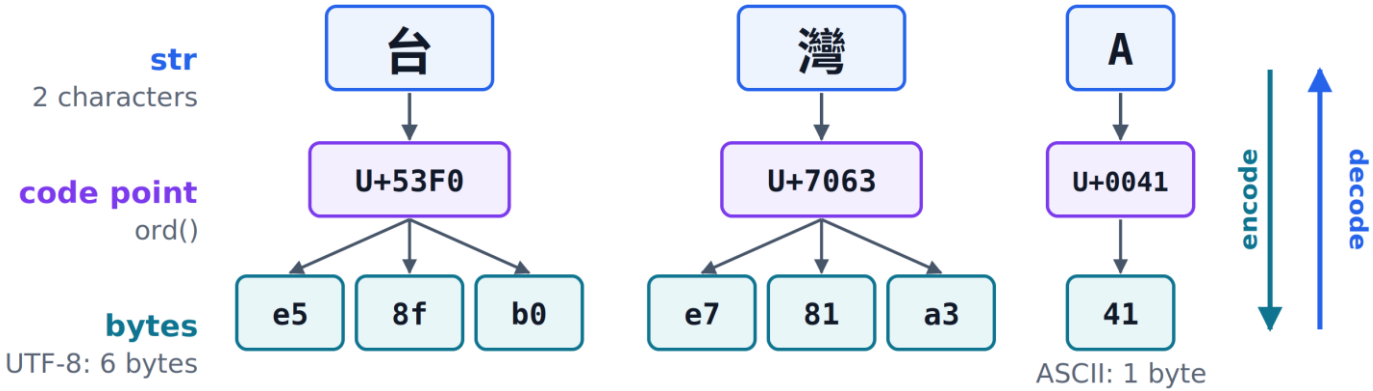
```c
char s[] = "hi";          // {'h', 'i', '\0'}: 3 bytes
s[0] = 'H';               // OK: C arrays are mutable
```

Python

```python
s = "hi"
print(len(s), type(s[0]))    # → 2 <class 'str'>
s[0] = "H"                   # → TypeError
s = "H" + s[1:]              # build a new string
print(s)                     # → Hi
```

Unicode, Bytes, and Encoding

- str holds characters (code points); bytes holds raw 8-bit values
- UTF-8: 1 byte for ASCII, 3 bytes for most Chinese characters
- C strlen() and C++ std::string::size() count bytes, not characters

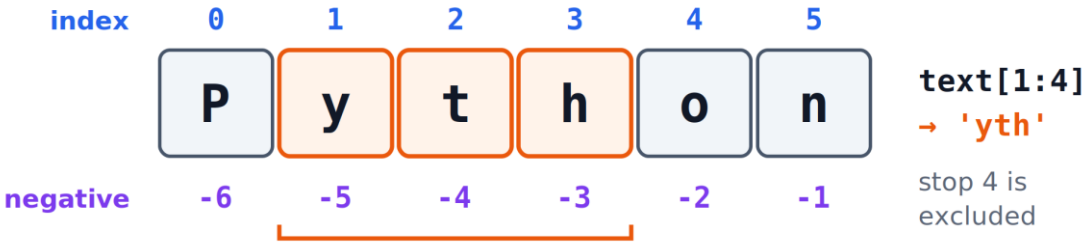



```
s = "台灣"
b = s.encode("utf-8")
print(len(s), len(b))            # → 2 6
print(b)                         # → b'\xe5\x8f\xb0\xe7\x81\xa3'
print(b.decode("utf-8") == s)    # → True
print(hex(ord("台")))            # → 0x53f0
```

String Indexing and Slicing

- String Indexing

```
# Positive indexing
text = "Python"
print(text[0])     # 'P' - First character
print(text[5])     # 'n' - Last character
# Negative indexing
print(text[-1])    # 'n' - First from the end
print(text[-6])    # 'P' - Sixth from the end
```



- Key Points
  - Strings are immutable
  - Indexing starts from 0
  - Negative indexing starts from -1 (last character)
  - Out of range index causes IndexError

String Slicing in Detail

```
string[start:stop:step]
```

- Examples and Applications

```
text = "Python Programming"

# Basic slicing
print(text[0:6])   # 'Python' - Index 0 to 5
print(text[7:])    # 'Programming' - From index 7 to end
print(text[:6])    # 'Python' - From start to index 5
print(text[:])     # 'Python Programming' - Complete string

# Step slicing
print(text[::2])   # 'Pto rgamn' - Every other character
print(text[1::2])  # 'yhnPormig' - From index 1, every other
print(text[::-1])  # 'gnimmargorP nohtyP' - Reverse string
```

String Slicing in Detail

```
# Practical examples
email = "user@example.com"
username = email[:email.index('@')] # 'user'
domain = email[email.index('@')+1:]          # 'example.com'

# Get file extension
filename = "document.pdf"
extension = filename[filename.rfind('.')+1:]  # 'pdf'
```

- Use Cases
  - Extract substrings
  - Reverse strings
  - Data parsing
  - Format processing

String Methods - Conversion

- Case Conversion

```
text = "Python Programming"
# Case conversion methods
print(text.upper())              # 'PYTHON PROGRAMMING' - All uppercase
print(text.lower())              # 'python programming' - All lowercase
print(text.capitalize())         # 'Python programming' - First letter capitalized
print(text.title())              # 'Python Programming' - Title case
print(text.swapcase())           # 'pYTHON pROGRAMMING' - Swap case

# Practical example
name = "john doe"
print(name.title())              # 'John Doe' - Standardize name format
# Case-insensitive comparison
password = "PyThOn123"
if password.lower() == "python123":
        print("Password matches!")
```

String Methods - Searching

- Search and Check Methods

```
text = "Python is awesome, Python is powerful"

# Finding substrings
print(text.find("Python"))          # 0 - First occurrence position
print(text.find("Python", 10))      # 19 - Search from index 10
print(text.find("Java"))            # -1 - Returns -1 if not found
print(text.index("Python"))         # 0 - Similar to find, but raises error if not found
# print(text.index("Java"))         # ValueError!

# Counting occurrences
print(text.count("Python"))         # 2 - "Python" appears 2 times
print(text.count("is"))             # 2 - "is" appears 2 times
```

String Methods - Searching

- Search and Check Methods

```
# Checking start and end
email = "user@example.com"
print(email.startswith("user"))     # True
print(email.endswith(".com"))       # True
print(email.endswith(".org"))       # False

# Practical example
url = "https://www.example.com"
if url.startswith("https://"):
        print("Secure connection")
elif url.startswith("http://"):
        print("Unsecure connection")
```

String Methods - Modification

- Modification and Cleaning Methods

```
# Replace
text = "Hello World, Hello Python"
print(text.replace("Hello", "Hi"))          # 'Hi World, Hi Python'
print(text.replace("Hello", "Hi", 1))       # 'Hi World, Hello Python' - Replace only once

# Strip whitespace
messy = " Python "
print(messy.strip())                # 'Python' - Remove whitespace from both ends
print(messy.lstrip())               # 'Python ' - Remove left whitespace
print(messy.rstrip())               # ' Python' - Remove right whitespace

# Strip specific characters
url = "https://www.example.com/"
print(url.strip("https://"))        # 'www.example.com'  !? (see next slide)
print(url.rstrip("/"))              # 'https://www.example.com'
```

Guess the Output: strip()

- We want to drop the "https://" prefix…

```
url = "https://shop.example.com/"
print(url.strip("https://"))    # ?
```

Answer **op.example.com**

strip() removes any of the characters {h, t, p, s, :, /} from both ends, not the prefix. Use url.removeprefix("https://").

Cleaning Text

- strip() removes whitespace (or a set of characters) from both ends
- removeprefix() / removesuffix() remove an exact string (3.9+)

```
raw = "   Alice@NTOU.edu.tw \n"
print(repr(raw.strip()))   # → 'Alice@NTOU.edu.tw'
email = raw.strip().lower()
print(email)               # → alice@ntou.edu.tw
url = "https://shop.example.com/"
print(url.removeprefix("https://"))   # → shop.example.com/
print(url.removesuffix("/"))   # → https://shop.example.com
print("a-b-c".replace("-", "+", 1))   # → a+b-c
```

split() and join()

- split() with no argument splits on any run of whitespace
- join() needs strings: convert numbers first

```
print("a  b c".split())         # → ['a', 'b', 'c']
print("a  b c".split(" "))      # → ['a', '', 'b', 'c']
print("k=v=w".split("=", 1))    # → ['k', 'v=w']
print("l1\nl2".splitlines())    # → ['l1', 'l2']
print("-".join(["2026", "10", "04"]))    # → 2026-10-04
nums = [1, 2, 3]
print(",".join(map(str, nums)))    # → 1,2,3
",".join(nums)                          # → TypeError
```

partition(): Split Once, Safely

- Always returns three parts, even when the separator is missing
- Safer than slicing with index(), which raises ValueError

```
email = "user@example.com"
name, sep, domain = email.partition("@")
print(name, domain)                    # → user example.com
print("no-at-sign".partition("@"))     # → ('no-at-sign', '', '')
key, _, value = "timeout = 30".partition("=")
print(key.strip(), int(value))         # → timeout 30
```

Is It a Number?

- The three checks disagree on Unicode digits, and all reject "-5" and "3.14"
- To validate a number, try int() / float() instead

string s	isdecimal()	isdigit()	isnumeric()	int(s)
"123"	✓	✓	✓	123
"３" full-width	✓	✓	✓	3
"²" superscript	✗	✓	✓	ValueError
"四"	✗	✗	✓	ValueError
"-5"	✗	✗	✗	-5
"3.14"	✗	✗	✗	ValueError

```
def to_int(s):
    try:
        return int(s)
    except ValueError:
        return None
print(to_int("-5"), to_int("3.14"), to_int("３"))    # → -5 None 3
```

Putting It Together: Validating Input

- Combine checks with any() and a generator expression

```
def check_username(u):
    if not u.isalnum():
        return "letters and digits only"
    if not 3 <= len(u) <= 20:
        return "length must be 3-20"
    return "ok"
def strength(pw):                # how many kinds of characters
    kinds = [any(c.isupper() for c in pw),
             any(c.islower() for c in pw),
             any(c.isdigit() for c in pw),
             any(not c.isalnum() for c in pw)]
    return sum(kinds)            # True counts as 1
print(check_username("user@1"))    # → letters and digits only
print(strength("Pass123!"))        # → 4
```

String Methods at a Glance

Task	Methods	Example	Result
Case	`upper() lower() casefold()`	`"Hi".upper()`	`'HI'`
Search	`in find() index() count()`	`"banana".count("an")`	`2`
Test	`startswith() endswith()`	`"a.py".endswith(".py")`	`True`
Test	`isdigit() isalnum()`	`"abc1".isalnum()`	`True`
Clean	`strip() lstrip() rstrip()`	`"  hi  ".strip()`	`'hi'`
Clean	`removeprefix() replace()`	`"v1".removeprefix("v")`	`'1'`
Split	`split() partition()`	`"a,b".split(",")`	`['a', 'b']`
Join	`sep.join(iterable)`	`"-".join("abc")`	`'a-b-c'`
Align	`center() ljust() zfill()`	`"7".zfill(3)`	`'007'`

String Formatting

Three Ways to Format: Use f-strings

- % comes from C's printf; you will still see it in older code
- f-strings (3.6+) are shortest and fastest, and can hold expressions
- The part after ':' is the same format spec in format() and f-strings

C

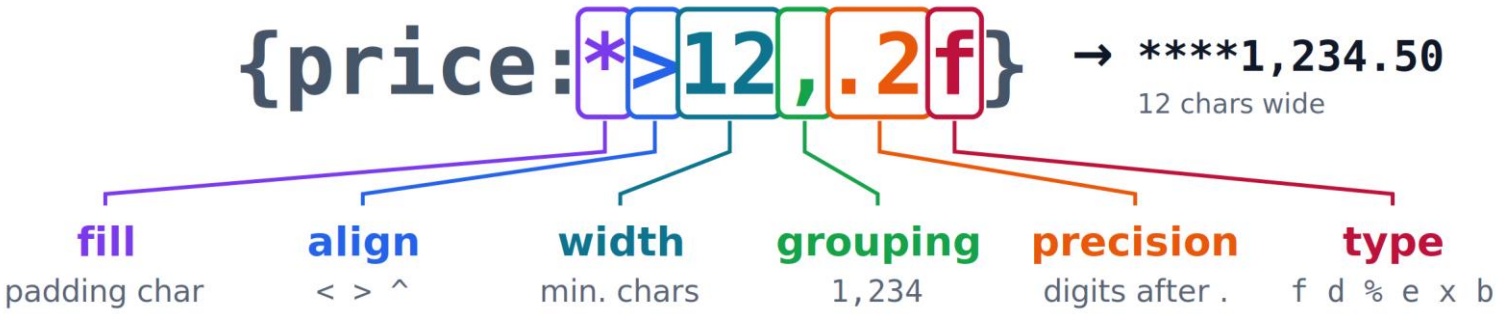
```c
printf("%s scored %.1f\n", name, score);
```

Python

```python
name, score = "Ann", 92.456
print("%s scored %.1f" % (name, score))
print("{} scored {:.1f}".format(name, score))
print(f"{name} scored {score:.1f}")
# → Ann scored 92.5   (all three lines print this)
```

Anatomy of a Format Spec

- Inside {value:spec}, every part is optional and they always come in this order




```
price = 1234.5
print(f"[{price:*>12,.2f}]")    # → [****1,234.50]
print(f"[{price:<12.1f}]")      # → [1234.5      ]
print(f"[{42:05d}]")            # → [00042]
```

Formatting Numbers

- Precision, separators, percent, scientific, other bases, signs

```
x = 1234567.891
print(f"{x:,.2f}")              # → 1,234,567.89
print(f"{0.856:.1%}")           # → 85.6%
print(f"{x:.3e}")               # → 1.235e+06
print(f"{255:b} {255:o}")       # → 11111111 377
print(f"{255:x} {255:#x}")      # → ff 0xff
print(f"{7:+d} {-7:+d}")        # → +7 -7
print(f"{2.5:.0f} {3.5:.0f}")   # → 2 4
```

Aligning Columns

- < left, > right, ^ center; strings default to left, numbers to right

```
items = [("Apple", 1.2, 10), ("Banana", 0.5, 125)]
print(f"{'Item':<8}{'Price':>8}{'Qty':>6}")
for name, price, qty in items:
    print(f"{name:<8}{price:>8.2f}{qty:>6}")
# → Item        Price   Qty
# → Apple        1.20    10
# → Banana       0.50   125
print(f"[{'hi':^10}] [{'hi':-^10}]")
# → [    hi    ] [----hi----]
```

f-string Power Features

- Any expression, = for debugging (3.8+), nested width, !r, dates

```
x, w = 42, 8
print(f"{x * 2 = }")              # → x * 2 = 84
print(f"{x=}")                    # → x=42
print(f"[{x:>{w}}]")              # → [      42]
name = "Ann"
print(f"{name!r}")                # → 'Ann'
from datetime import date
d = date(2026, 10, 4)
print(f"{d:%Y/%m/%d}")            # → 2026/10/04
print(f"{'pass' if x >= 60 else 'fail'}")   # → fail
```

Practical: A Text Progress Bar

- String repetition and f-strings build simple terminal output

```
done, total, width = 45, 100, 20
ratio = done / total
filled = int(width * ratio)
bar = "" * filled + "" * (width - filled)
print(f"|{bar}| {ratio:.0%} ({done}/{total})")
# → |                                        | 45% (45/100)
```

Summary

- Dict: a hash table; keys must be hashable; get / pop for safe access
  - Counter, defaultdict, dispatch tables, dict-by-ID indexes
- Strings: immutable Unicode; methods return new strings
  - encode / decode between str and bytes; removeprefix, not strip
- Formatting: f-strings and the format spec {value:fill align width , .precision type}

Q&A Session

- Any Questions?
- Open Discussion

